BIGO

Most Important topic SDE Interview
Google Fcb Alibaba Amazon

- Also termed as BIGO Asymptotic Analysis

How well a problem is solved → BIGO

? What is Good Code ?

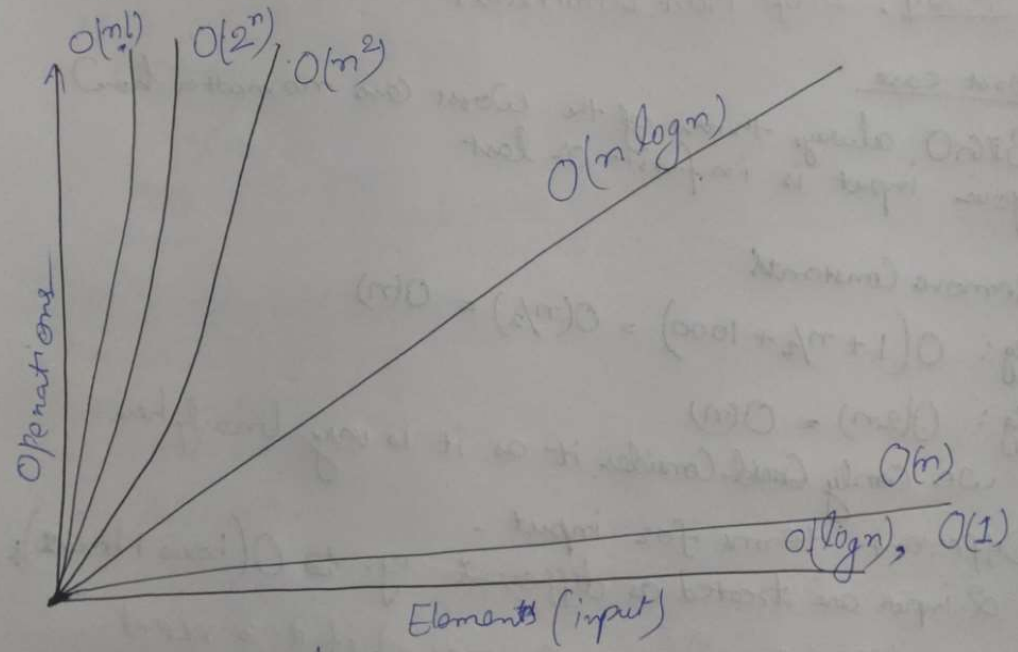
1. Readable – can understand your code
2. Scalable → BIG O gives idea that Code is Scalable or not?

Speed
Memory
Scalable ? → measure ability to increase or decrease in performance wrt some features.

BIGO language we use to talking about how long an algorithm takes to run.

Big O Complexity Chart

input ↑ How much algorithm slowdown



~~The slower the algorithm~~

How many operation do we have to do as we increases the input i.e. BIGO.

O(n) - Linear time

Input ↑ ~~then~~ operations also increasing then Graph is linear

O(1) Constant - time

$O(4+7n) = O(n)$

$O(3+4n) = O(n)$

How?

→ Rules Book

Rule 1: Worst Case

Rule 2: Remove Constants

Rule 3: Different terms for inputs

Rule 4: Drop Non-Dominants

→ Worst case

BIGO always thinks of the worst case no matter how your input is in first or last

→ Remove Constants

eg: $O(1 + n/2 + 1000) = O(n/2) = O(n)$

eg: $O(2n) = O(n)$

we ~~only~~ ~~could~~ consider it as it is vary linearly here.

→ Different terms for input.

2 inputs are treated as different eg. ~~to~~ O(boxes + boxes2)

or may be O(boxes * boxes2) if it is nested

→ ~~Rule~~ Drop Non-Dominants

$O(x^2 + 3x + 100 + x/2)$

$= O(x^2)$

we dont care about insignificant ~~digits~~ terms

Data Structure + Algorithms = Programs

Store data

function / way to use data structure

Right Data structure

Right Algorithm

O(n!) → The waste BIG O, terrible!

eg. We are adding every input in nested loop

you are never going to see it.

? Which code is best ?

Readable | Memory → space-complexity | Speed → Time complexity

Program (memory)

- Heap (Variable)
- Stack (function call();)

→ Aux Space Complexity

- Variable
- Data Structure
- function call
- Allocation

BHASKARKUMAR5 / InnocentFrequentElec...

Run

Invite

script.js

```javascript
//#1 -- For loop in Javascript.
const nemo = ['nemo'];

function findNemo1(array) {
  for (let i = 0; i < array.length; i++) {
    if (array[i] === 'nemo') {
      console.log('Found NEMO!');
    }
  }
}

findNemo1(nemo);
```

Console Shell

```
Found NEMO!
```

script.js

```javascript

// const nemo = ['nemo','dorry'];
const large=new Array(100).fill('nemo');  //fill all 100 item created with nemo

function findNemo1(array) {
  let t0= performance.now(); //for starting timer
  for (let i = 0; i < array.length; i++) {
    if (array[i] === 'nemo') {
      console.log('Found NEMO!');
    }
  }
  let t1= performance.now(); //checking timer at this moment
  console.log('call to find Nemo took '+ (t1-t0)+ '  milliseconds');
}

// findNemo1(nemo);
findNemo1(large);
```

Console Shell

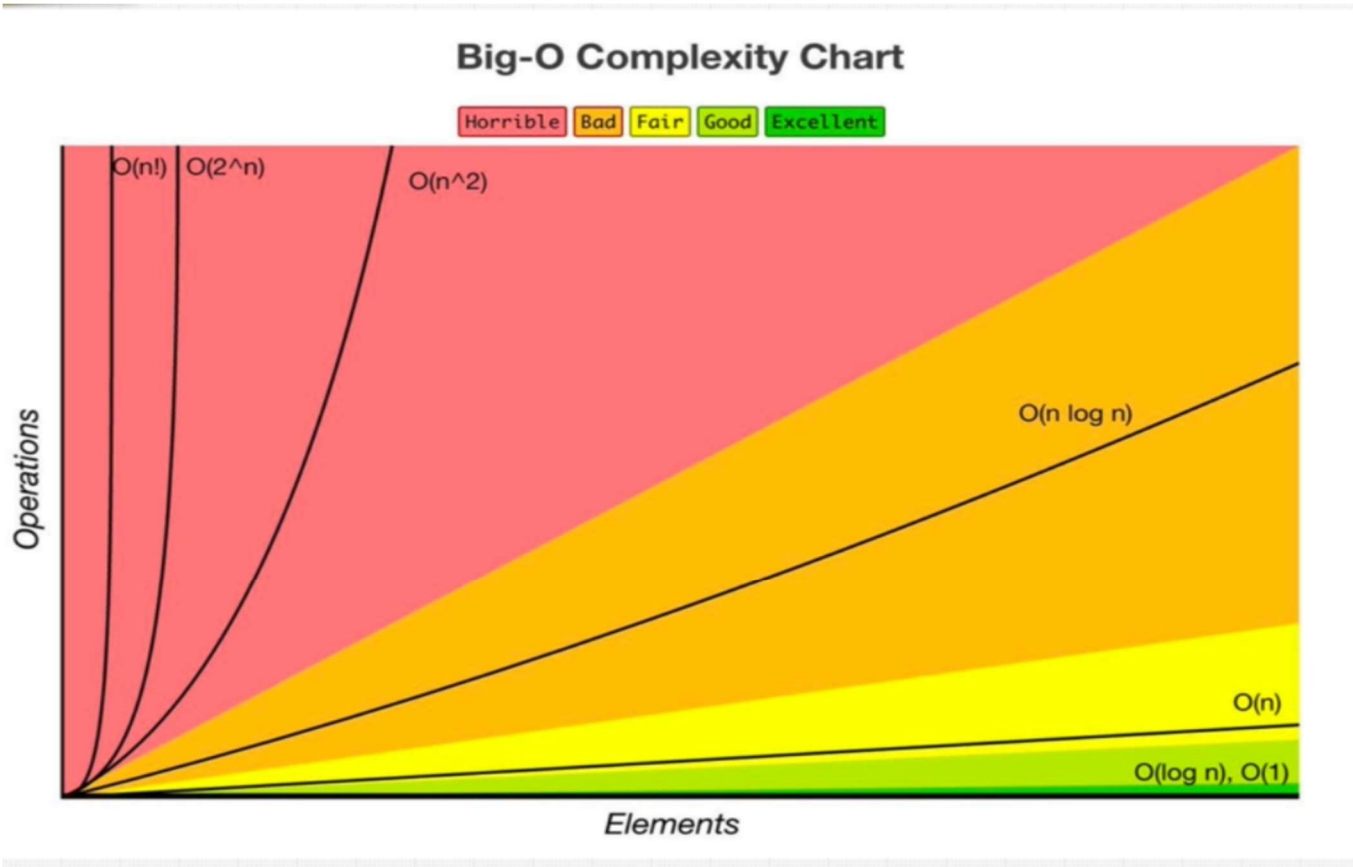
```
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
call to find Nemo took 23.899999998509884  milliseconds
```

script.js

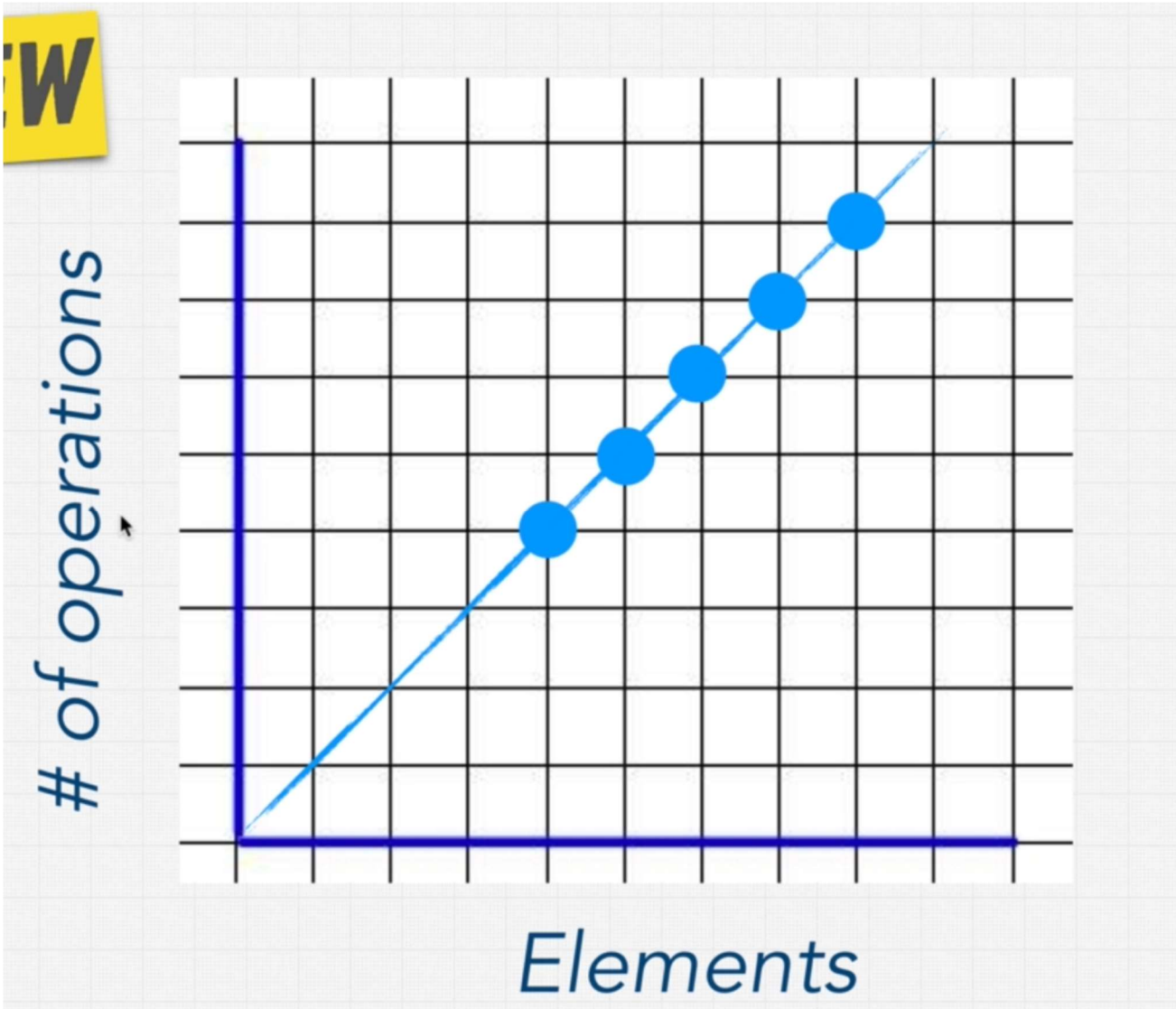
```javascript

// const nemo = ['nemo','dorry'];
const large=new Array(100000).fill('nemo');  //fill all 100 item created with nemo

function findNemo1(array) {
  let t0= performance.now(); //for starting timer
  for (let i = 0; i < array.length; i++) {
    if (array[i] === 'nemo') {
      console.log('Found NEMO!');
    }
  }
  let t1= performance.now(); //checking timer let t0: number
  console.log('call to find Nemo took '+ (t1-t0)+ '  milliseconds');
}

// findNemo1(nemo);
findNemo1(large);
```

Console Shell

```
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
Found NEMO!
call to find Nemo took 3581.89999999851  milliseconds
>
```

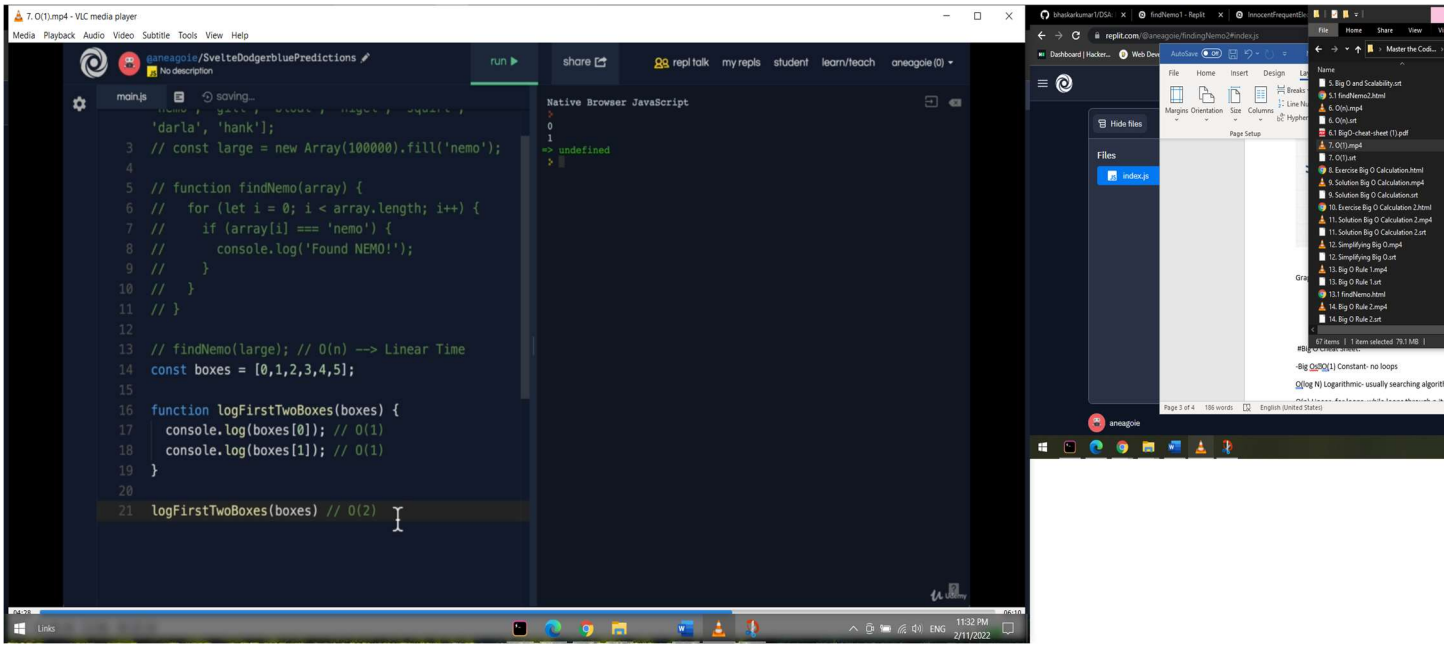


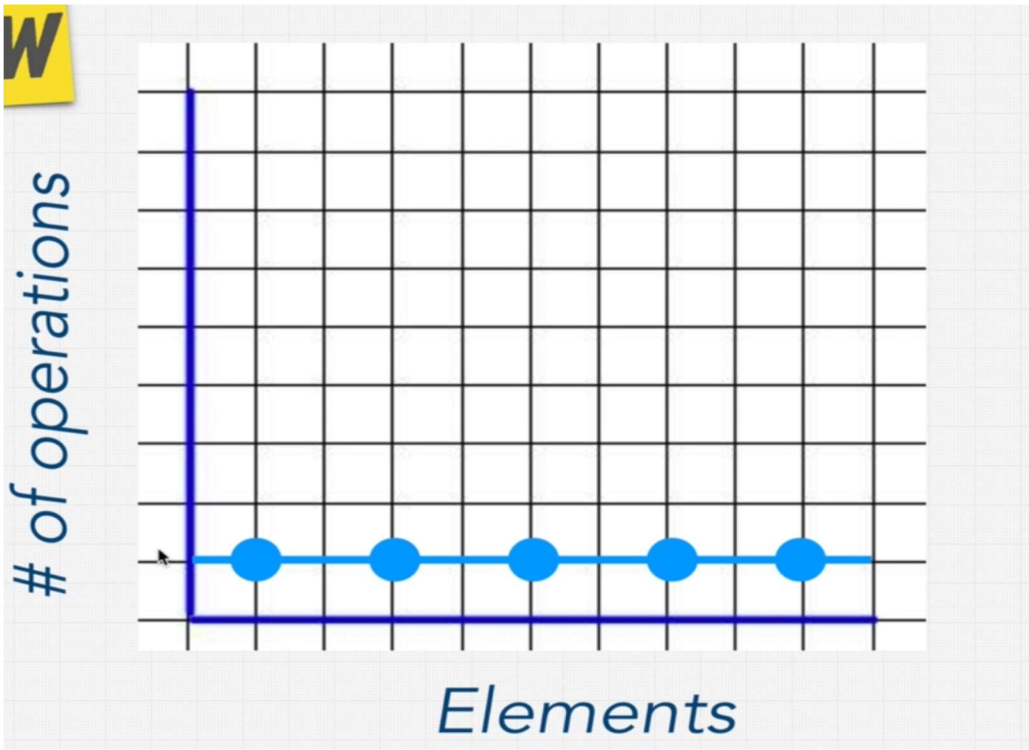
NEMO eg.



Graph vary linearly therefore it’s BIG0 is O(n) =linear time , n= no. of inputs

O(1) =constant time





#Big O Cheat Sheet:

-Big Os-

O(1) Constant- no loops
O(log N) Logarithmic- usually searching algorithms have log n if they are sorted (Binary Search)
O(n) Linear- for loops, while loops through n items
O(n log(n)) Log Liniear- usually sorting operations
O(n^2) Quadratic- every element in a collection needs to be compared to ever other element. Two nested loops
O(2^n) Exponential- recursive algorithms that solves a problem of size N
O(n!) Factorial- you are adding a loop for every element

Iterating through half a collection is still O(n)
Two separate collections: O(a * b)

-What can cause time in a function?-

Operations (+, -, *, /)
Comparisons (<, >, ==)
Looping (for, while)
Outside Function call (function())

-Rule Book-

Rule 1: Always worst Case
Rule 2: Remove Constants
Rule 3: Different inputs should have different variables. O(a+b). A and B arrays nested would be O(a*b)
+ for steps in order
* for nested steps
Rule 4: Drop Non-dominant terms

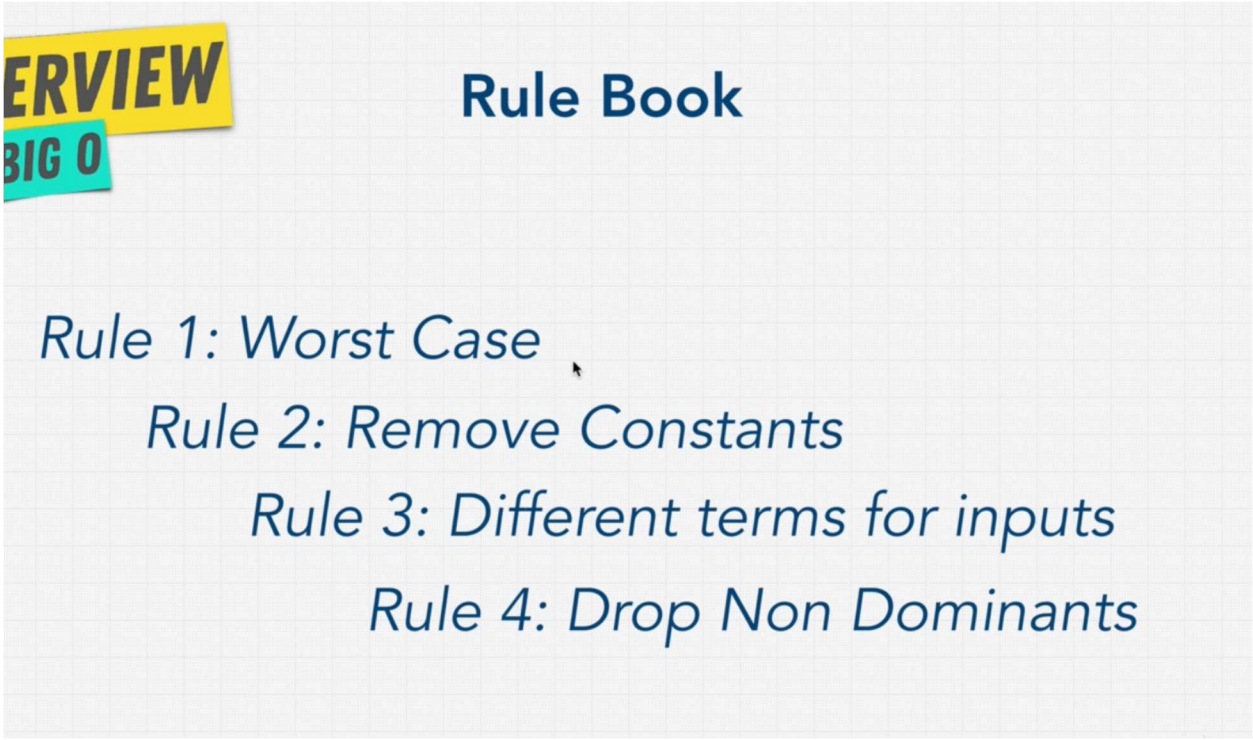
-What causes Space complexity?-

Variables
Data Structures
Function Call
Allocations

O(4+7n)=O(n)

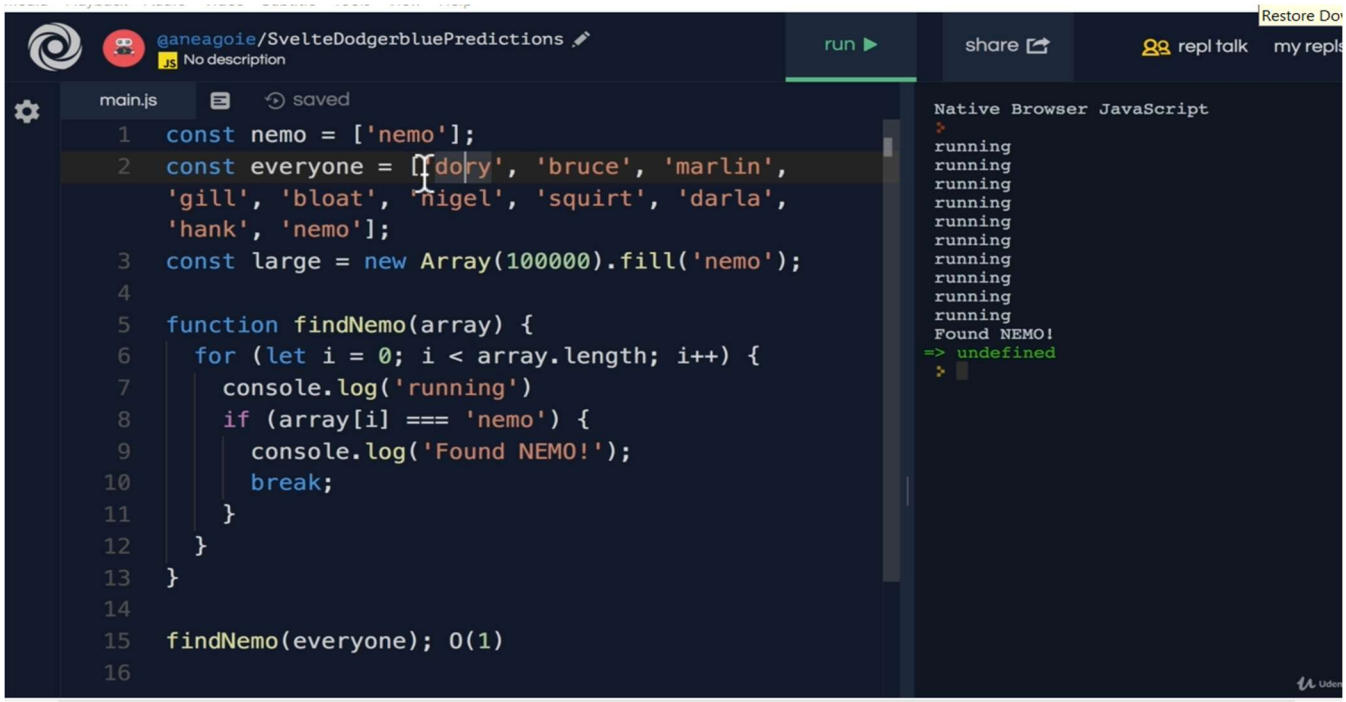
O(3+4n)= O(n)

HOW?



1. Rule

Worst Case



the worst case is nemo is in last

```
const nemo = ['nemo'];
const everyone = ['dory', 'bruce', 'marlin', 'gill', 'bloat', 'nigel', 'squirt', 'darla', 'hank', 'nemo'];
const large = new Array
function findNemo(array
  for (let i = 0; i < a
    console.log('runnin
    if (array[i] === 'n
      console.log('Foun
      break;
    }
  }
}
findNemo(everyone); O(n)
```

```
Native Browser JavaScript
running
running
running
running
running
running
```

Rule 2. Remove Constant

Eg. O(1+n/2+10000)= O(n/2)=O(n)

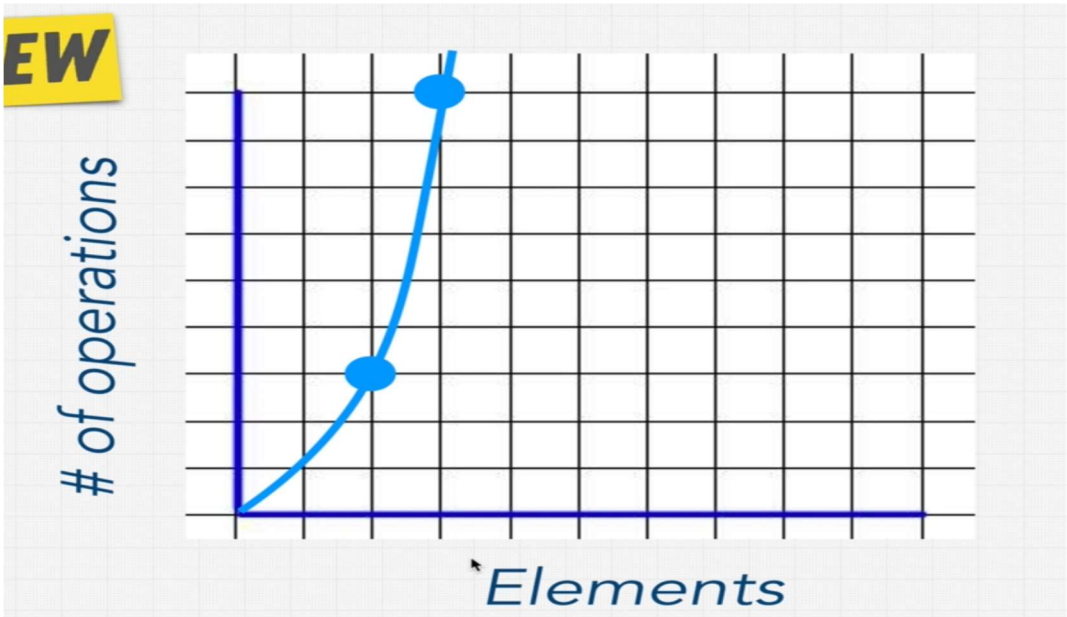
Eg. O(2n)=O(n)

Rule 3. Different terms for input

```
function compressBoxesTwice(boxes, boxes2) {
    boxes.forEach(function(boxes) {
        console.log(boxes);
    });


    boxes2.forEach(function(boxes) {
        console.log(boxes);
    });
}

O(a + b)
```

The above BIGO will be O(boxes +boxes2) and it will O(boxes*boxes2) if they get nested loop

O(n^2)


EW
# of operations
Elements

main.js
saved
// Log all pairs of array
const boxes = ['a','b','c','d','e'];
function logAllPairsOfArray(array) {
for (let i = 0; i < array.length; i++) {
for (let j = 0; j < array.length; j++) {
console.log(array[i], array[j])
}
}
}
logAllPairsOfArray(boxes)
O(n^2)
Native Browser JavaScript
a a
a b
a c
a d
a e
b a
b b
b c
b d
b e
c a
c b
c c
c d
c e
d a
d b
d c
d d
d e
e a
e b
e c
e d
e e
=> undefined

Rule 4.

Drop NON- dopinant terms

sumofallpairs.js

```javascript
// print the all elementand then sum of all pairs numbers of arrays
function printAllnumbersthenAlllPairSum(arrays) {
console.log("these are numbers: ");
arrays.forEach(function(array){
console.log(array);
});


console.log('these are there sum: ');
arrays.forEach(function(firstNumber){
  arrays.forEach(function(secondNumber){
    console.log(firstNumber+secondNumber);
  });
});
}

printAllnumbersthenAlllPairSum([0,1,2,3,4,5,6,7,8,9])

// O(n+ n^2)
//  O(n^2)
```

17 Answers

Active | Oldest | Score

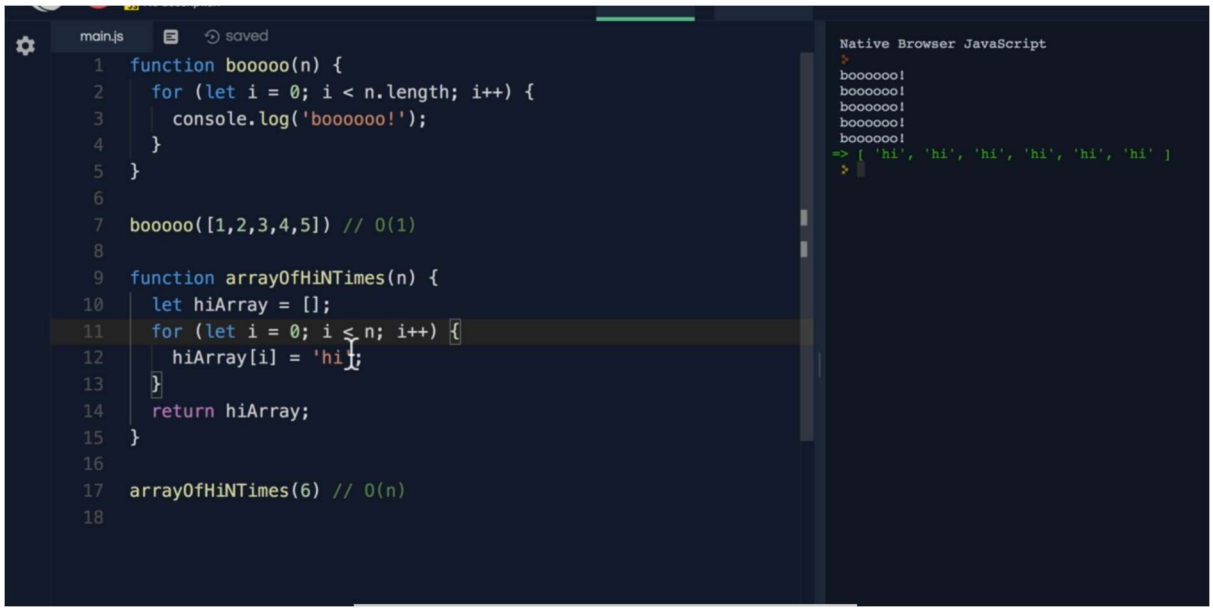
120

There you go. This is probably the most trivial example of a function that runs in `O(n!)` time (where `n` is the argument to the function):

```
void nFacRuntimeFunc(int n) {
  for(int i=0; i<n; i++) {
    nFacRuntimeFunc(n-1);
  }
}
```

Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)

Space Complexibility

Eg.